Checkpoint – Streaming Pipeline Phase 1

Infrastructure, Simulator, Profile Hydration, Enrichment Service

Yassine Ben Jemia
Direction Centrale du Système d'Information

June 26, 2026

Contents

1 Overview **2**

2 Infrastructure **2**
2.1 Docker Compose 2
2.2 Centralized Configuration 2

3 Event Simulator **2**

4 Profile Hydration **3**
4.1 Redis Key Schema 3

5 Enrichment Service **3**
5.1 Feature Categories (30 features) 4
5.2 Buffer Update Strategy 4

6 Streaming Architecture Concepts **4**

7 Validation Results **5**

8 Project Structure Update **5**

9 Next Phase **5**

1 Overview

This checkpoint covers the first phase of the streaming pipeline implementation, transitioning the anomaly detection platform from batch-only processing to a real-time architecture. The session established the containerized infrastructure, implemented the event simulator, built the profile hydration pipeline, and delivered a working Enrichment Service consuming from RedPanda.

2 Infrastructure

2.1 Docker Compose

Two services are containerized via `docker-compose.yml`:

Service	Port(s)	Notes
RedPanda	9092, 8082, 9644	Kafka-compatible broker. Dev mode: `-smp 1`, `-memory 512M`, `-overprovisioned`. Dual listener: `PLAINTEXT://redpanda:29092` (inter-container), `OUTSIDE://localhost:9092` (host).
Redis 7	6379	Alpine image. Hot profile store and event buffer persistence.

Both services use named Docker volumes (`redpanda-data`, `redis-data`) for data persistence across container restarts. PostgreSQL remains on the host (localhost:5432) pending full containerization.

2.2 Centralized Configuration

A `.env` file at the project root stores all connection parameters (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `REDIS_HOST`, `REDIS_PORT`, `KAFKA_BOOTSTRAP`). The module `src/config.py` loads these via `python-dotenv` and exposes `DB_CONFIG`, `REDIS_HOST`, `REDIS_PORT`, and `KAFKA_BOOTSTRAP` for import by any service.

Files migrated to centralized config: `src/streaming/simulator.py`, `src/batch/hydrate_redis.py`. Remaining files (`scripts/load_postgresql.py`, `src/batch/batch_profile.py`, `src/training/data_prep.py`) to be updated on next touch.

3 Event Simulator

Attribute	Detail
File	`src/streaming/simulator.py`
Input	PostgreSQL `transactions` table, ordered by timestamp ASC
Output	`raw-events` RedPanda topic
Partition key	`client_id` (preserves per-client ordering for LSTM)
Replay mode	Respects inter-event gaps divided by `speed_factor` (default 100). Sleep capped at 2.0s to skip large gaps (weekends).
Burst mode	Fixed delay between events (`burst_delay_ms`, default 10ms). For development and testing.

Oracle stripping	`is_anomaly` and `anomaly_type` excluded from published messages — raw events contain only observable facts.
Flush strategy	`producer.flush()` every 1000 events for responsiveness.

Validation: 500 events published and consumed back successfully. Partition keys verified as `client_id` UUIDs. Message payloads confirmed to contain only the 11 raw event envelope fields.

4 Profile Hydration

Attribute	**Detail**
File	`src/batch/hydrate_redis.py`
Classification	Batch job (runs before streaming pipeline starts)
Source	PostgreSQL: `profile_snapshots` (latest per client), `employee_profile_snapshots` (latest per employee), `archetype_baselines`
Destination	Redis key-value store

4.1 Redis Key Schema

Colon-separated namespacing convention for key organization and category-level operations (`KEYS profile:client:*`, etc.):

Pattern	**Content**	**Count**
`profile:client:{id}`	Client profile JSONB snapshot	10,000
`profile:employee:{id}`	Employee profile JSONB snapshot	300
`baseline:{archetype}`	Archetype baseline statistics	5
`buffer:client:{id}`	Rolling event buffer (max 50)	Dynamic
`buffer:employee:{id}`	Rolling employee buffer (max 100)	Dynamic

Production scheduling: runs as a pre-shift cache warming step (e.g., 07:30) after the nightly batch profile rebuild completes. In the Airflow DAG (weeks 3–4), this becomes a downstream task of the profile rebuild DAG.

5 Enrichment Service

Attribute	**Detail**
File	`src/streaming/enrichment_service.py`
Consumes	`raw-events` topic
Produces	`enriched-events` topic
Group ID	`enrichment-service` (offset tracking + horizontal scaling)
Profile source	Redis (`profile:client:{id}`) — static + batch stats
Buffer source	Redis (`buffer:client:{id}`, `buffer:employee:{id}`)

Feature count	30 contrast features (identical to batch enrichment logic)

5.1 Feature Categories (30 features)

Category	Count	Source
Raw event (amount, hour, op one-hot, branch match)	7	Event
Amount contrast (z-score, ratio, thresholds)	6	Event + Profile
Operation contrast	1	Event + Profile
Timing contrast	2	Event + Profile
Counterparty contrast	1	Event payload
Velocity (24h/7d counts, cumulative, duplicates, smurfing)	5	Redis buffer
Employee (tx count, pair count)	2	Redis buffer
Context (archetype one-hot, account age)	6	Profile

5.2 Buffer Update Strategy

Hybrid approach combining per-event Redis updates with nightly batch rebuilds:

Update Type	Fields	Rationale
Per-event (Redis)	`recent_events_buffer`, client/employee buffers	Small, append-only, critical for velocity features
Nightly batch (PostgreSQL)	`amount_mean/std`, `op_mix`, distributions, z-scores	Require full history for accurate computation

Buffer persistence in Redis ensures survival across service restarts. If Redis is flushed, the nightly batch reconstructs all buffers from the PostgreSQL `transactions` table — consistent with the hot/cold architecture principle that Redis is fast but expendable.

6 Streaming Architecture Concepts

Key Kafka/RedPanda concepts applied in this phase:

Concept	Application
Producer/Consumer	Each middle service (Enrichment, Scoring, Decision) is both. Topics between every stage for auditability and fault isolation.
Partition key	`client_id` ensures per-client event ordering across all operations (required for LSTM sequence model).
Consumer group	Enables offset tracking (resume after restart) and horizontal scaling (partitions split across group members).
Serialization	JSON with `value_serializer`/`value_deserializer`. `key_serializer` encodes `client_id` as UTF-8 bytes.
Poll loop	`while True` with `poll(timeout_ms=1000)` — service never exits, individual polls have short timeouts.
Advertised listeners	Dual listener config: internal (`redpanda:29092`) for inter-container, external (`localhost:9092`) for host.

7 Validation Results

Test	Result
RedPanda connectivity	Producer/consumer round-trip verified
Simulator (500 events)	All published to `raw-events`, keys = client_id, no oracle fields
Profile hydration	10,000 clients + 300 employees + 5 baselines loaded
Enrichment Service	500 events consumed, enriched, produced to `enriched-events`. No errors.

8 Project Structure Update

New files created this session:

```
docker-compose.yml
.env
src/config.py
src/streaming/__init__.py
src/streaming/simulator.py
src/streaming/enrichment_service.py
src/batch/hydrate_redis.py
scripts/test_connection.py
scripts/consumer_test.py
```

9 Next Phase

The next session completes the streaming pipeline:

1. **Scoring Service** — consume from `enriched-events`, run AE + LSTM inference via `ScoreFusion`, produce to `scored-events`.
2. **Decision Service** — consume from `scored-events`, apply regulatory rules and tier assignment, produce to `decisions`.
3. **End-to-end test** — full pipeline: simulator → enrichment → scoring → decision.
4. **Streamlit dashboard** — consume from `decisions` topic.